

Inference-Time Orchestration

Dynamic LoRA Routing for Orthogonal ETFT Calibration

By Lukas Benda

Architectural Blueprint · April 2026

Abstract

The Orthogonal Calibration strategy requires combining a "Depth" model (historical context) with "Width" models (generational creativity) during inference. Rather than utilizing a computationally prohibitive sparse Mixture of Experts (MoE) architecture—which requires loading multiple massive expert networks simultaneously—we propose a **Dynamic LoRA (Low-Rank Adaptation) Routing Engine**. In this paradigm, a single generalized Foundation Model serves as the Depth base, while generational Width constraints are hot-swapped into VRAM as lightweight LoRA adapters precisely when the generation epoch requires lateral creativity.

1 Why MoE is Suboptimal for GPU-Poor ETFT

A traditional Mixture of Experts (MoE) routing layer (e.g., Switch Transformer architecture) analyzes incoming tokens and routes them to specific feed-forward networks (Experts).

- **The VRAM Bottleneck:** An MoE model containing 8 experts requires the VRAM to hold all 8 experts simultaneously. If we have 10 evolutionary generations, storing 10 Width models is impossible on consumer hardware.
- **Token-Level Confusion:** MoE routes at the token level. In architectural design, switching logic mid-block (e.g., routing one line of PyTorch to the a_1 expert and the next line to the a_3 expert) leads to syntactical corruption. We need *Task-Level* routing, not token-level.

2 The Dynamic LoRA Routing Architecture

Low-Rank Adaptation (LoRA) freezes the pre-trained weights of the model and injects trainable rank decomposition matrices. In our architecture, the Foundation Model (The Depth Historian) remains static in VRAM. The specialized Width creativity is encoded entirely within tiny (10–50 MB) LoRA files.

2.1 The Inference Flow

When the Orchestrator initiates the synthesis of a_{n+1} (SOTA+1), the process operates in three phases:

1. Phase 1: The Depth Analysis (Base Model Only)

The Orchestrator feeds the historical trajectory ($a_0 \rightarrow a_n$) into the unaltered Foundation Model. The model outputs a structural analysis (e.g., *"The bottleneck in a_n is the memory footprint of self-attention"*).

2. Phase 2: Agentic Retrieval

Based on the depth analysis, the system retrieves relevant scientific papers (e.g., Linear Attention, FlashAttention).

3. Phase 3: The Width Synthesis (Hot-Swapped LoRA)

The Orchestrator dynamically loads the specific **Generational Width LoRA (L_n)** onto the Foundation Model. This LoRA adapter is trained specifically to maximize lateral architectural variance for generation n . The system now synthesizes the code.

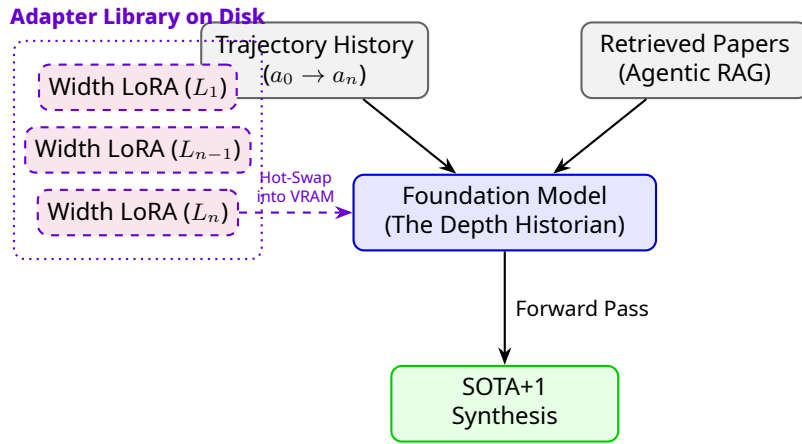


Figure 1: Dynamic LoRA Routing. The heavy Foundation Model (Depth) remains in VRAM. Lightweight Generational Adapters (Width) are hot-swapped based on the required evolutionary epoch, enabling multi-agent specialization without the VRAM penalties of MoE.

3 The Auto-Regressive SOTA+2 Challenge

This architecture elegantly solves the recursive SOTA+2 problem. If the system successfully validates SOTA+1 in the physical sandbox, it can *train a new LoRA adapter on the fly* (L_{n+1}) based on the lateral variance discovered during the SOTA+1 generation. This new LoRA is saved to the disk library and subsequently hot-swapped onto the Foundation model to initiate the SOTA+2 synthesis loop.

4 Conclusion

Dynamic LoRA Routing provides the strict task-level orchestration required for Orthogonal Calibration. It preserves the "GPU-Poor" philosophy by minimizing active VRAM footprint, while allowing the ETFT system to dynamically switch between deep historical context and wide lateral creativity at the exact moment of architectural synthesis.